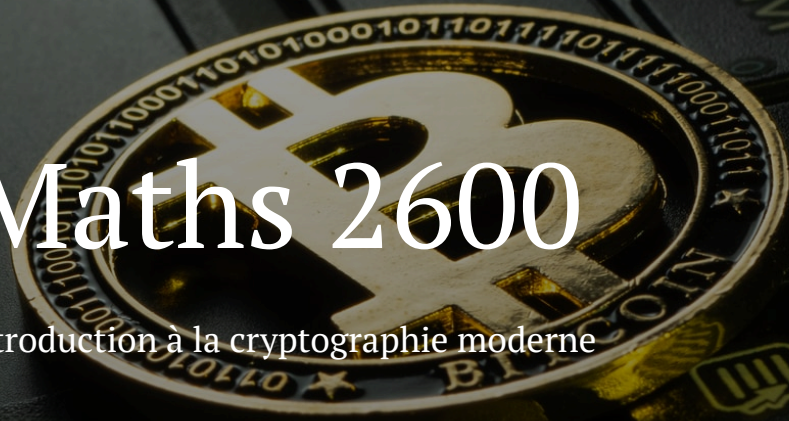


Maths 2600

Introduction à la cryptographie moderne



Un peu d'histoire

- Communication a distance avec le code Morse
- Cryptographie pendant la guerre: Machine Enigma
 - Utilisé par les allemands
 - Fonctionnement mécanique avec des rotors configurable, à placer selon un livre de clefs
 - Turing a été l'un des principaux contributeurs au cassage d'Enigma
- Internet et réseaux mondiaux: nécessite d'avoir du chiffrement sécurisé dans des environnements à grand nombre d'acteurs

Au delà du chiffrement affine: Chiffrement de Hill

- Exemple sur: https://www.wikiwand.com/fr/Chiffre_de_Hill
- Etabli en 1929
- Combine arithmétique modulaire et algèbre linéaire
- Multiplication matricielle
- On groupe les caractères à chiffrer en vecteurs, on multiplie par la matrice
 - => mélange les caractères entre eux, ce n'est pas une simple transposition
 - Generalise le chiffrement affine à plusieurs symboles consécutifs
 - La matrice doit avoir une inverse, on l'utilise pour déchiffrer
 - Un premier pas vers des maths plus avancées pour chiffrer
- Mais vulnérable: peut être cassé via l'analyse de suffisamment de séquences clair <-> chiffré

La cryptographie moderne

- Depuis l'avènement des ordinateurs, puis d'internet, on utilise une cryptographie plus avancée mathématiquement
- Basée sur la théorie des nombres, la complexité de certains calculs sur des nombres premiers, l'arithmétique modulaire, la théorie des groupes
- Dans ce cadre, tous les messages sont codé en binaire
 - Donc un message est un nombre
 - Si le message a chiffrer est trop long pour un algorithme donné, on le découpe en messages de taille fixe, qui sont tous des nombres
 - A partir de là on peut faire de l'arithmétique sur des grands nombres
- Primitives et protocoles
 - Il peut être difficile de s'y retrouver: beaucoup de variantes et standards
- Reference: le livre Real-World Cryptography de David Wong

Primitives

- Ce sont les blocs de base qu'on pourra combiner pour construire d'autres primitives, ou des protocoles:
 - Fonctions de hachage cryptographique
 - MAC (Message authentication codes), des primitives pour authentifier un message
 - Le chiffrement authentifié symétrique
 - Echange de clefs
 - Chiffrement asymétrique et hybride
 - Signatures cryptographiques
 - Preuves à non divulgation (Zero Knowledge Proofs)
 - L'aléatoire

Protocoles

- Des algorithmes plus complets basé sur les primitives à haute sécurité pour atteindre un résultat donné dans un contexte ou il peut y avoir des attaquants
 - Transport de données chiffrés: SSL, TLS
 - Chiffrement de bout en bout
 - Authentification d'utilisateurs
 - Réseaux décentralisés: Bittorent, Bitcoin, Ethereum, ZeroNet
- Un protocole peut avoir des failles via l'utilisation qu'il fait de ses primitives, même si celle ci sont sécurisées

Authentification et intégrité

- Dans un contexte avec beaucoup de participants (internet par exemple), on s'intéresse à deux propriétés de sécurité:
 - Authentification: l'entité à qui je parle est-elle la vraie ou un usurpateur ?
 - Intégrité: est-ce que le message que je reçois a été modifié par un attaquant ?
- Exemples: site de paiement, serveurs d'authentification à un réseau social, messagerie

Primitives cryptographiques

Sécurité des primitives

- On a deux manières d'éprouver une primitive:
- On se base sur un problème mathématique dont la solution est longue à calculer (exemple: RSA, Diffie-Hellman)
- On se base sur de l'analyse statistique, des heuristiques, et le travail des cryptanalystes (si personne n'arrive à casser une primitive pendant plusieurs années, on la considère sécurisée. Exemple: AES pour le chiffrement symétrique)

Fonctions de hachage cryptographique

- Fonction de hachage: permet d'assigner un identifiant entre 0 et N à tous les éléments d'un ensemble (souvent plus large)
 - En crypto l'ensemble de départ c'est les messages binaires (de taille arbitraire), donc des nombres entiers de 0 à l'infini
 - N est en général très large, par exemple 2^{256} pour des hash codés sur 256 bits
 - En général on écrit les hash en hexadécimal ou base64, exemple:
f63e68ac0bf052ae923c03f5b12aedc6cca49874c1c9b0ccf3f39b662d1f487b
 - 64 caractères hexa = $64 * 4 \text{ bits} = 256 \text{ bits}$
- La sortie d'une fonction de hachage est souvent appelée un digest

Utilité des fonctions de hachage

- En cryptographie, les fonctions de hachage sont très utiles, quelques exemples:

Utilité des fonctions de hachage

- Engagement
 - Imaginez que vous êtes au courant de quelque chose sans vouloir le révéler, mais vous voulez prouver après coup que vous étiez au courant
 - Par exemple: les résultats d'une entreprise, le score aux élections présidentielles, jouer un pion caché dans un jeu en ligne, ...
 - Vous pouvez calculer un hash de cette information (exprimé comme un message "à la date DD/MM/YYYY, je sais que ...") et le donner à vos amis
 - Le jour où vous révélez ce que vous saviez en donnant le message, ils peuvent calculer son hash et vérifier que c'est bien celui que vous leur aviez fourni

Utilité des fonctions de hachage

- Intégrité des données numériques
 - Lorsque vous téléchargez un fichier, vous voulez être certain qu'il n'a pas été modifié par un attaquant
 - Exemple: un exécutable, un code source inclut
 - Si vous possédez un hash de la donnée, fourni par une entité de confiance, vous pouvez calculer le hash de la donnée que vous télécharger et comparer
 - Note: le navigateur peut le faire automatiquement pour les scripts javascript inclut depuis des sites externes

Utilité des fonctions de hachage

- BitTorrent
 - Les fichiers à partager sont découpés en chunks
 - Le hash de chaque chunk est utilisé pour l'identifier de manière efficace
 - Un peer peut ainsi savoir les chunks qu'il a ou pas, et les demander aux autres peer qui annonce avoir les chunks manquant

Propriétés de sécurité

- Une fonction de hachage doit respecter un certain nombre de propriétés assurant sa sécurité dans un cadre cryptographique
 - Pre-image resistance
 - Second pre-image resistance
 - Collision resistance
- Ces propriétés suppose que le message en entrée est de taille suffisamment grande, et que l'espace de valeurs de la fonction est très large
- Un exemple de fonction de hachage triviale qui **n'a pas ces propriétés**:
 - $f(x) = x \mod 2^{256}$

Pre-image resistance

- Inverser la fonction de hachage doit être difficile: je ne dois pas pouvoir retrouver un message d'origine si je n'ai que son hash
 - On dit que la fonction est à sens unique (one-wayness)
- Exemple sur la fonction triviale: Si j'ai un hash h tel que $h = x \bmod 2^{256}$, je sais qu'il existe k tel que $x = k \times 2^{256} + h$, je peux donc facilement trouver x (un message original) qui a pour hash h .
 - TP: trouver 3 messages (des nombres) ayant pour hash
115792089237316195423570985008687907853269979211514109474003138754790027457785
 - Donc la fonction triviale n'a pas la propriété de Pre-image resistance
- Cette propriété permet d'utiliser le hashage pour les engagements
 - Je ne peux pas facilement calculer le message original supposé secret

Second pre-image resistance

- Connaître un message d'entrée et son hash ne doit pas permettre de trouver facilement un autre message avec le même hash
- Exemple sur la fonction triviale: Je connais h et x tel que $h = x \bmod 2^{256}$. Je peux construire $x + 2^{256}$ qui aura le même hash.
 - Donc la fonction triviale n'a pas la propriété de Second pre-image resistance
- Cette propriété permet d'utiliser le hashage pour l'intégrité des données
 - Je ne peux pas facilement calculer une autre donnée possédant le même hash que je pourrais substituer à l'originale

Collision resistance

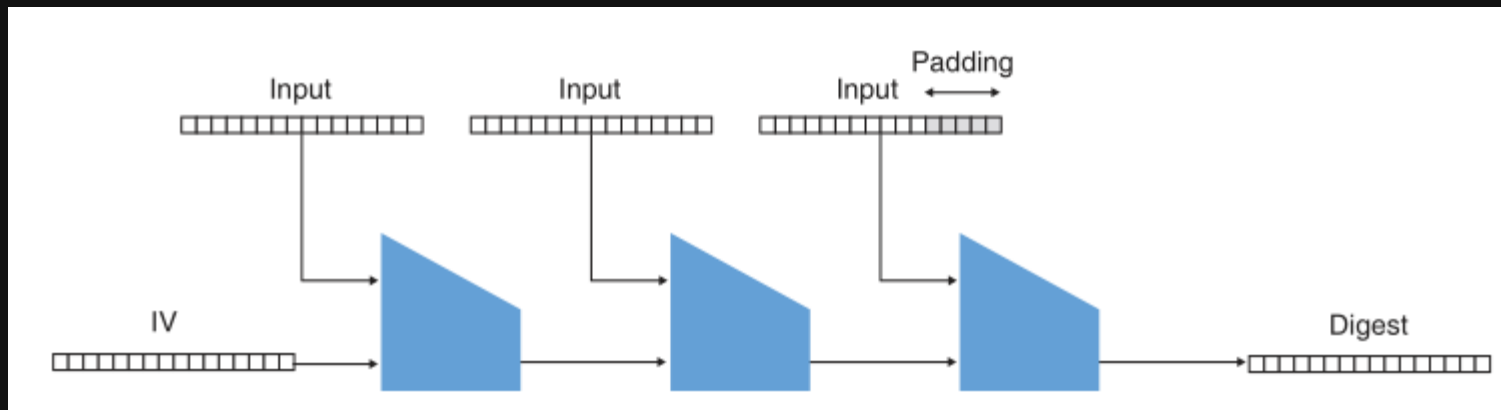
- Il doit être difficile de générer deux messages d'entrée donnant le même hash
- Exemple sur la fonction triviale: Les messages $x + 2^{256}$, $x + 2 \times 2^{256}$, $x + 3 \times 2^{256}$, ... donnent tous le même hash
 - Donc la fonction triviale n'a pas la propriété de Collision resistance
- Cette propriété permet d'utiliser le hashage pour identifier de manière unique des données
 - Dans le cadre de BitTorrent, j'ai très peu de chance de tomber sur deux chunks différent ayant le même hash

Fonctions de hashage standard

- Dans les années 90: MD5 et SHA-1
 - MD5: cassée en 2004
 - SHA-1: cassée en 2016
 - N'ont pas la collision resistance
 - Toujours utilisés dans certaines applications
- Etat de l'art: SHA-2 et SHA-3

SHA-2

- Créé par la NSA, standardisé en 2001
- Variantes: SHA-224, SHA-256, SHA-384, SHA-512
- Basée sur un enchainement de blocs:
 - A chaque étape applique un chiffrement et un XOR (ou exclusif), itérativement sur le message découpé
 - Chaque bloc produit un hash intermediaire de taille fixe, la dernier hash produit est le résultat



- Vulnérable aux attaques par extension

SHA-3

- Compétition débutée en 2007, 64 candidats, pour trouver un nouveau standard de hashage résistant aux attaques par extension
- 5 ans plus tard: la fonction Keccak est nommée et devient SHA-3
- Variantes: SHA-3-224, SHA-3-256, SHA-3-384, SHA-3-512
- Basée sur des blocs de permutations
- Extensions: SHAKE et cSHAKE
 - SHAKE: similaire à SHA-3, avec taille de sortie arbitraire
 - cSHAKE: SHAKE configurable par une clef

Hashage en python ou avec openssl

```
import hashlib
```

```
msg = b"hello world"
print("SHA224 Hash: ", hashlib.sha224(msg).hexdigest())
print("SHA256 Hash: ", hashlib.sha256(msg).hexdigest())
print("SHA384 Hash: ", hashlib.sha384(msg).hexdigest())
print("SHA512 Hash: ", hashlib.sha512(msg).hexdigest())
print("SHA3-224 Hash: ", hashlib.sha3_224(msg).hexdigest())
print("SHA3-256 Hash: ", hashlib.sha3_256(msg).hexdigest())
print("SHA3-384 Hash: ", hashlib.sha3_384(msg).hexdigest())
print("SHA3-512 Hash: ", hashlib.sha3_512(msg).hexdigest())
```

```
MSG="hello world"
echo -n $MSG | openssl dgst -sha224
echo -n $MSG | openssl dgst -sha256
echo -n $MSG | openssl dgst -sha384
echo -n $MSG | openssl dgst -sha512
echo -n $MSG | openssl dgst -sha3-224
echo -n $MSG | openssl dgst -sha3-256
echo -n $MSG | openssl dgst -sha3-384
echo -n $MSG | openssl dgst -sha3-512
```

Hashage de mots de passe

- En base de donnée on stocke des hash des mot de passe utilisateurs
- En général on combine le mot de passe avec un nombre aléatoire (salt) pour chaque utilisateur et stocké en clair
 - Permet d'éviter les attaques par dictionnaire de mots de passe
 - Protège les utilisateurs qui utilisent les même mot de passe sur différents sites
- Inscription:
 - Alice s'inscrit avec son password `p`
 - Le serveur génère aléatoirement `salt` et calcule `h = hash(p + salt)`
 - Le serveur stocke `h` et `salt` pour Alice
- Connexion:
 - Alice se connecte avec son password `p`
 - Le serveur récupère le `h` et `salt` de Alice et calcule `h2 = hash(p + salt)`
 - Si `h2 == h` alors le serveur authentifie Alice

TP: attaque par dictionnaire et protection

- L'objectif de ce TP est d'attaquer une base de donnée de hash de mots de passe non salté à l'aide d'une attaque par dictionnaire
- Les fonctions à compléter se trouvent dans le fichier `src/cryptoy/passwords.py`
- Le script contient un code de base qui:
 - Définit deux fonctions utilitaires:
 - `hash_password` utilise `hashlib.sha3_256` pour calculer le hash d'un mot de passe
 - `random_salt` génère un salt aléatoire
 - Définit la fonction `generate_users_and_password_hashes` que j'ai utilisé pour générer la base de donnée stockée dans le fichier `tests/data/passwords-database.json`
 - Définit les fonctions à implémenter pour passer les tests

TP: attaque par dictionnaire et protection

1. Attaque

- Implémentez la fonction `attack` pour calculer les mot de passe en clair
- Cette fonction prend en paramètre la liste complète des mot de passe possibles, et la base de donnée associant à chaque utilisateur un hash de mot de passe
- Elle doit renvoyer un dictionnaire avec pour clefs les noms d'utilisateurs et pour valeur leur mot de passe en clair
- Le test associé est `test_passwords_attack` , celui ci doit passer suite à votre implémentation

2. Fix

- Implémentez la fonction `fix` qui doit calculer une nouvelle base de donnée avec des password hashé avec un salt
- La fonction doit renvoyer un dictionnaire avec pour clefs les noms d'utilisateurs et pour valeur un dictionnaire ayant pour champs `password_hash` et `password_salt` .
- Implémentez la fonction `authenticate` qui doit renvoyer `True` si un couple `(user, password)` est correct, `False` sinon.

Message Authentication Codes (MAC)

- Permet d'authentifier un utilisateur au cours d'un échange répété de messages
 - Exemple classique: les cookie, pour éviter d'avoir à entrer son username / password à chaque requête ou le stocker en clair dans le navigateur
- Protocole:
 - L'utilisateur s'authentifie via son username + password
 - Le serveur envoie à l'utilisateur un MAC: `(username, HASH(secret_key, username))` (la `secret_key` étant connue du serveur uniquement)
 - Pour les requêtes suivantes, l'utilisateur envoie son MAC
- Vulnérabilité:
 - la fuite du MAC
 - la fuite de la clef secrète
 - les temps de calcul: le serveur doit vérifier les MAC en temps constant (pas d'early return)
 - Sinon un attaquant peut se baser sur ce temps pour construire des MAC en modifiant octet par octet et en vérifiant les temps de vérification

Chiffrement symétrique

- Confidentialité des messages
- Basé sur une clef secrète partagée qui sert au chiffrement et au déchiffrement
 - Le chiffrement de César est un chiffrement symétrique
- Le standard: Advanced Encryption Standard (AES)
 - 3 versions en fonction de la taille de la clef AES-128, AES-192 et AES-256
 - Fonctionne par blocks de bits sur le message d'entrée, qui font des permutations dans un espace gigantesque
 - Chaque clef produit une permutation différente
- On utilise souvent un **nonce** indexant les messages de manière unique au sein d'un échange pour éviter les replay attack

TP: Chiffrement symétrique en python

- La lib `cryptography` propose différents algorithmes de chiffrement, dont AES:

- `from cryptography.hazmat.primitives.ciphers.aead import AESGCM`

- Cette classe s'instancie avec une clef de 128, 192 ou 256 bits

- Les methodes `encrypt(nonce: bytes, msg: bytes, None)` et `decrypt(nonce: byte, msg: bytes, None)` permettent de chiffrer et déchiffrer

- On peut utiliser la fonction `from secrets import token_bytes` pour calculer un nonce en lui passant le nombre d'octets voulu

- Implémentez les fonction `encrypt` et `decrypt` du fichier `src/cryptoy/aes_cipher.py`

- Déchiffrez le message

`b'\xd0\x8d)%\x18QnD\xfb\x9c\xc7(\x1a\x85\xc3t\xf3\xc4\x92"\x1ahB\xfb\xfb\xa1\xc1]\xee\xfb\xda\xba`

`?\xb8\xe1\xb4{\x87\n2'` avec la clef `b"al\yourbasearebelongtous"` et le nonce

`b'\xfa}_\xe1\x9cN\x0cz/\xebNt'` en lançant le test `test_aes()` après avoir implémenté vos fonctions:

- `python -m pytest -k test_aes -s`

L'échange de clefs

- Un problème fondamental de la cryptographie est l'échange de clefs (pour du chiffrement symétrique, ou du hash configurable)
- Comment Alice et Bob peuvent se mettre d'accord sur une clef à utiliser, sans que cette clef ne puisse être connue d'un attaquant ?
- Deux solutions classiques:
 - Construction via l'algorithme de Diffie-Hellman d'une clef secrète partagée
 - Partage d'une clef secrète via chiffrement asymétrique comme RSA
- Attention: sans authentification un échange de clef n'est pas sécurisé
 - Vulnérable à une man in the middle attack

Diffie-Hellman

- Publié en 1976 dans l'article "New Direction in Cryptography", c'est la première méthode connue qui utilise un problème de math algorithmique avancé pour la cryptographie
- Ce problème est le logarithme discret en arithmétique modulaire
 - Ce problème est trop coûteux à calculer pour être attaqué
 - On utilise les propriétés mathématique du logarithme discret pour opérer un partage de clef qui sera secrète car trop longue à calculer sans connaitre les paramètres secrets
- Plus généralement, on peut appliquer Diffie-Hellman en s'appuyant sur des groupes finis dans lesquels le problème du logarithme discret est coûteux

Structures mathématiques

- L'étude des structures mathématique est une branche de l'algèbre
- Une structure mathématique est en général un ensemble muni d'opérations respectant certaines propriétés
- On étudie ce type de structure pour généraliser des démonstrations à tous les ensemble respectant une structure donnée
 - Un peu comme en programmation orientée objet, on généralise un ensemble de classes par une interface
- Une des structure de base est celle de **groupe**

La théorie des groupes

- En mathématique, un groupe est un ensemble munit d'un opérateur binaire respectant certaines propriétés
- On dit que $(G, \circ)$ est un groupe si:
 - $\circ$ est un opérateur binaire sur l'ensemble G : $\forall x, y \in G, x \circ y \in G$
 - Il existe un élément neutre pour $\circ$: $\exists e \in G, \forall x \in G, x \circ e = e \circ x = x$
 - Tout élément a un inverse pour $\circ$: $\forall x \in G, \exists x^{-1} \in G, x \circ x^{-1} = x^{-1} \circ x = e$
 - L'opérateur $\circ$ est associatif: $\forall x, y, z \in G, (x \circ y) \circ z = x \circ (y \circ z) = x \circ y \circ z$
- A partir de cette définition, on peut démontrer des propriétés générales sur tous les groupes
 - Par exemple on pourrait démontrer que l'élément neutre est unique dans n'importe quel groupe

Exemple de groupe: $(\mathbb{Z}, +)$

- L'ensemble des nombres entiers relatifs $\mathbb{Z}$ munit de l'addition $+$ est un groupe:
 - Pour tout $x \in \mathbb{Z}$ et $y \in \mathbb{Z}$, l'addition $x + y$ est bien dans $\mathbb{Z}$
 - L'élément neutre est 0, on a bien $x + 0 = 0 + x = x$ pour tout $x \in \mathbb{Z}$
 - Tout élément x a bien un inverse, c'est $-x$ car: $x + (-x) = (-x) + x = 0$
 - L'addition est bien associative: on peut additionner dans n'importe quel ordre, pas besoin de parenthèses

Exemple de groupe: $(\mathbb{Q}^*, \times)$

- L'ensemble des nombre rationnels privé de 0 (fractions) $\mathbb{Q}^*$ munit de la multiplication $\times$ est un groupe:
 - Pour tout $x \in \mathbb{Q}^*$ et $y \in \mathbb{Q}^*$, le produit $x \times y$ est bien dans $\mathbb{Q}^*$
 - L'élément neutre est 1, on a bien $x \times 1 = 1 \times x = x$
 - Tout élément x a bien un inverse, c'est $\frac{1}{x}$ car: $x \times \frac{1}{x} = \frac{1}{x} \times x = 1$
 - La multiplication est bien associative: on peut multiplier dans n'importe quel ordre, pas besoin de parenthèses

Exemple de groupe: $(\{0, 1\}, \oplus)$

- L'ensemble des booléen $\{0, 1\}$ munit du **ou exclusif** logique $\oplus$ est un groupe:
 - $0 \oplus 0 = 1 \oplus 1 = 0 \in \{0, 1\}$
 - $0 \oplus 1 = 1 \oplus 0 = 1 \in \{0, 1\}$
 - L'élément neutre est 0, on:
 - $0 \oplus 0 = 0$
 - $1 \oplus 0 = 1$
 - 0 est son propre inverse car $0 \oplus 0 = 0$
 - 1 est son propre inverse car $1 \oplus 1 = 0$
 - Le **ou exclusif** logique est bien associatif

Groupes et arithmétique modulaire

- On note $\mathbb{Z}/n\mathbb{Z}$ l'ensemble $\{0, 1, \dots, n - 1\}$ correspondant aux restes modulo n
- L'ensemble $\mathbb{Z}/n\mathbb{Z}$ munit de l'addition $+$ est un groupe
 - Par exemple dans: $\mathbb{Z}/3\mathbb{Z} = \{0, 1, 2\}$
 - 0 est l'élément neutre
 - 0 est l'inverse de 0 car $0 + 0 \bmod 3 = 0$
 - 1 est l'inverse de 2 car $1 + 2 \bmod 3 = 3 \bmod 3 = 0$

Groupe modulaire multiplicatif

- On peut se demander si $\mathbb{Z}^*/n\mathbb{Z}$ munit de la multiplication est un groupe
 - Par exemple dans $\mathbb{Z}^*/3\mathbb{Z} = \{1, 2\}$, 1 est l'élément neutre
 - 1 est l'inverse de 1 car $1 \times 1 \bmod 3 = 1$
 - 2 est l'inverse de 2 car $2 \times 2 \bmod 3 = 4 \bmod 3 = 1$
 - $\mathbb{Z}^*/4\mathbb{Z} = \{1, 2, 3\}$
 - On a $2 \times 2 \bmod 4 = 4 \bmod 4 = 0 \notin \mathbb{Z}^*/4\mathbb{Z}$
 - Donc $\times$ n'est pas un opérateur binaire sur cet ensemble, donc $(\mathbb{Z}^*/4\mathbb{Z}, \times)$ n'est pas un groupe
- On peut montrer que $(\mathbb{Z}^*/n\mathbb{Z}, \times)$ est un groupe ssi n est premier
- On peut tout de même construire un groupe en tant que sous ensemble de $\mathbb{Z}^*/n\mathbb{Z}$
 - Exemple: si on prend $\{1, 3\} \subset \mathbb{Z}^*/4\mathbb{Z}$ on obtient un groupe:
 - $1 \times 1 = 1 \bmod 4$
 - $1 \times 3 = 3 \bmod 4$
 - $3 \times 3 = 9 = 1 \bmod 4$ (3 est son propre inverse)

Le logarithme discret au sein d'un groupe

- On peut généraliser l'exponentiation a n'importe quel groupe, on pose alors:
 - $x^{\circ, n} = x \circ x \circ \dots \circ x$
- Si $a = x^{\circ, n}$, on dit que n est le logarithme de a pour l'opérateur $\circ$ et la base x , on note $\log_{x, \circ}(a) = n$
- En arithmétique modulaire modulo p premier, le calcul du logarithme pour la multiplication est compliqué
 - Si p est très grand, le calcul est trop long pour les machines modernes (centaines de milliers d'années)
 - Cela signifie que connaître a ne permet pas de calculer n
 - On peut donc imaginer publier a et garder n secret au sein d'un protocole

Le protocole d'échange de clefs de Diffie-Hellman

- Alice et Bob veulent calculer une clef secrète commune pour sécuriser leurs échanges
 - Cette clef privée ne doit pas être calculable par un espion observant l'échange
- Ils choisissent ensemble deux nombres: un très grand nombre premier p , et un générateur $g \in \{2, \dots, p - 1\}$
- Alice tire aléatoirement un nombre secret $a \in \{2, \dots, p - 1\}$ et calcule $A = g^a \mod p$
- Bob tire aléatoirement un nombre secret $b \in \{2, \dots, p - 1\}$ et calcule $B = g^b \mod p$
- Alice et Bob s'échangent publiquement A et B
- Alice reçoit B , elle calcule $B^a \mod p$. Or $B^a = (g^b)^a = g^{b \times a}$.
- Bob reçoit A , il calcule $A^b \mod p$. Or $A^b = (g^a)^b = g^{a \times b}$.
- On a donc $B^a \mod p = A^b \mod p$, Alice et Bob peuvent donc utiliser ce nombre comme clef pour leurs échanges
- Un espion qui intercepterait A et B ne peut pas connaître cette clef:
 - Pour la calculer il faut posséder a ou b , qui sont secret
 - Pour calculer a ou b , il faut calculer le logarithme discret, ce qui est trop long

Exponentiation modulaire rapide

- Les calcul de la forme $B^E \bmod M$ sont des exponentiation modulaire
- Pour calculer une exponentiation rapide (complexité en $O(\log(E))$):

```
def pow_mod(B, E, M):  
    if E == 0:  
        return 1  
    elif E == 1:  
        return B % M  
    else:  
        root = pow_mod(B, E // 2, M)  
        if E % 2 == 0:  
            return (root * root) % M  
        else:  
            return (root * root * B) % M
```

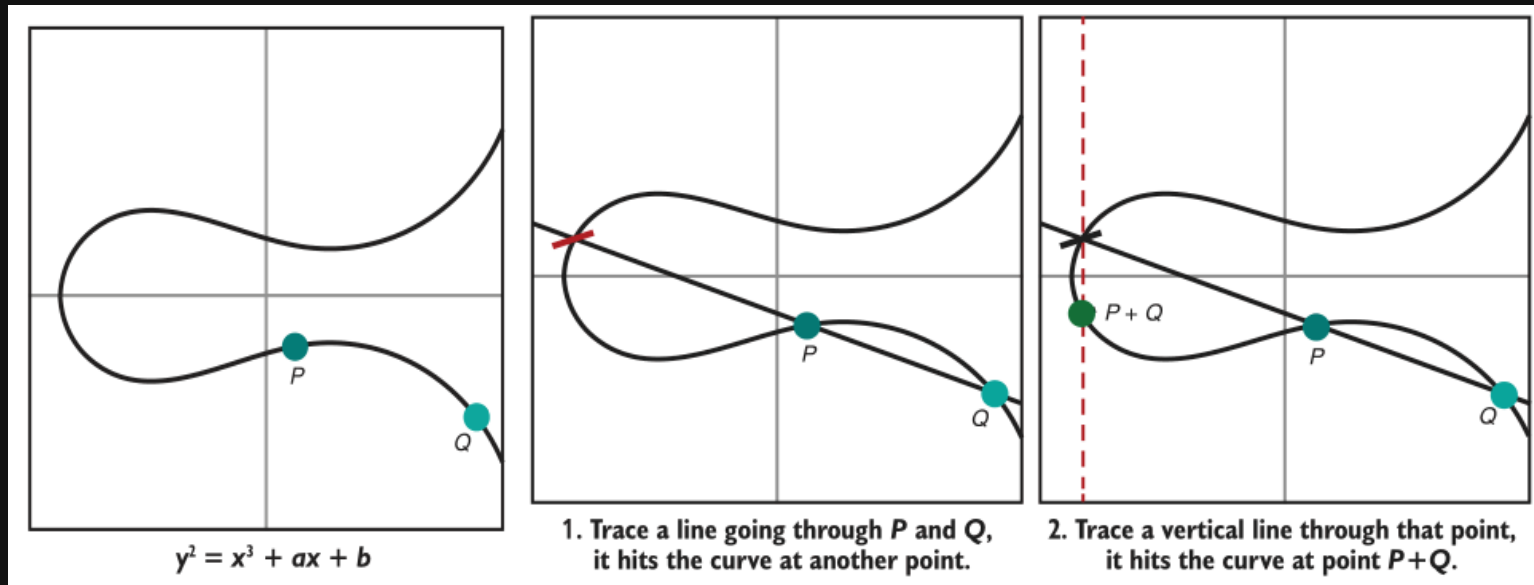
- La fonction est définie dans `src/cryptoy/utils.py`

TP: Implémenter Diffie-Hellman

- Dans le fichier `src/cryptoy/diffie_hellman.py` :
- Implémentez la fonction `def keygen(prime_number: int, generator: int) -> dict` en suivant les instructions en commentaire
- Implémentez la fonction `def compute_shared_secret_key(public: int, private: int, prime_number: int) -> int`
 - Celle ci doit renvoyer la clef partagée par Alice et Bob
 - Alice reçoit B , elle calcule $B^a \bmod p$. Or $B^a = (g^b)^a = g^{b \times a}$.
 - Bob reçoit A , il calcule $A^b \bmod p$. Or $A^b = (g^a)^b = g^{a \times b}$.
- Vérifiez que le test `def test_diffie_hellman()` passe. Celui ci orchestre un échange de clef entre deux participants Alice et Bob et montre comment utiliser la clef avec deux méthodes de chiffrement (Fernet et AES)
- Choisissez un binome, generez chacun une clef, partagez votre clef publique à l'autre, puis appelez la fonction `compute_shared_secret_key` et verifiez que vous obtenez bien la même clef secrète.

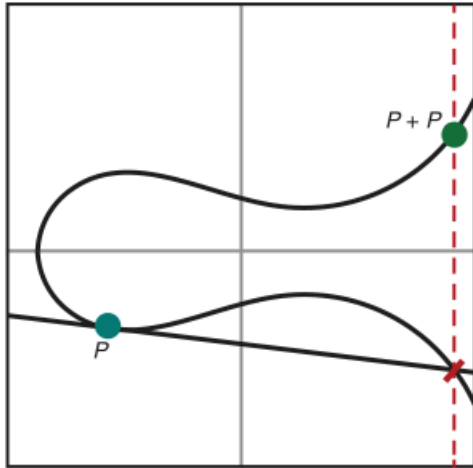
Diffie-Hellman et courbes elliptiques

- Le protocole de Diffie-Hellman peut s'appliquer dans d'autres groupes
- En particulier, le groupe des points d'une courbe elliptique est souvent utilisé en cryptographie

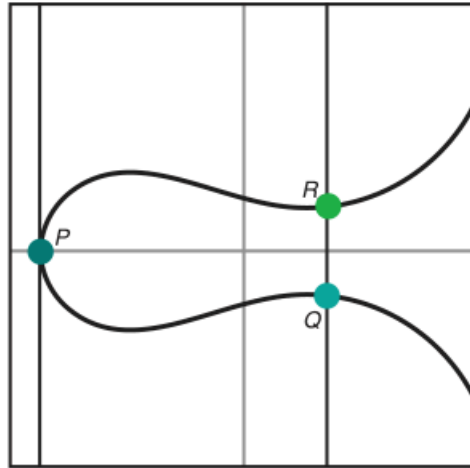


Diffie-Hellman et courbes elliptiques

- Element neutre: point à l'infini



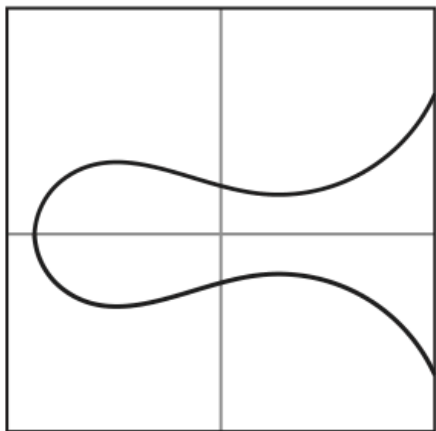
To add a point to itself, draw the tangent and follow the previous method.



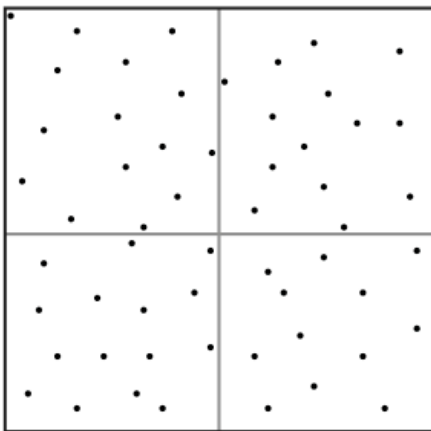
Sometimes adding points results in the point at infinity: $P + P = O$ and $R + Q = O$.

Diffie-Hellman et courbes elliptiques

- Combiné à l'arithmétique modulaire, on obtient des propriétés mathématiques permettant une complexité accrue avec des paramètres de taille plus faible



$$y^2 = x^3 + ax + b$$



$$y^2 = x^3 + ax + b \bmod p$$

Chiffrement asymétrique

- Basé sur une clef publique et une clef privée
 - Les autres utilisent ma clef publique pour chiffrer des messages qu'ils m'envoient
 - J'utiliser ma clef privée pour les déchiffrer
- Peut être coûteux et limité (en taille des messages)
 - Est donc souvent utilisé pour faire un échange de clef pour chiffrement symétrique ensuite
 - Standard: ECIES (Elliptic Curve Integrated Encryption Scheme)

RSA

- Publié en 1977, peu de temps après l'algorithme de Diffie-Hellman
- Conceptuellement proche, également basé sur de l'exponentiation modulaire difficile à inverser

RSA: Idée

- On veut chiffrer un message m représenté par un entier (representation binaire = entier)
- On se donne $n > m$ très grand et on considère $\mathbb{Z}^*/n\mathbb{Z} = 1, \dots, n-1$
- Les puissances de m modulo n créent un cycle: on peut trouver i tel que $m^i = m \pmod n$
 - Exemple: so $n = 5$ et $m = 2$, on a:
 - $m^2 = 2 * 2 = 4 \pmod 5$
 - $m^3 = 2 * 2 * 2 = 8 = 3 \pmod 5$
 - $m^4 = 2 * 2 * 2 * 2 = 16 \pmod 5 = 1 \pmod 5$
 - $m^5 = 2 * 2 * 2 * 2 * 2 = 32 \pmod 5 = 2 \pmod 5 = m$
- On peut donc imaginer chiffrer m par $M = m^e$
- Pour déchiffrer M , il faut connaître d tel que $M^d = (m^e)^d = m^{e \times d} = m \pmod n$
 - D'une certaine manière, e est la clef publique et d la clef privée
 - Il y a des contraintes mathématiques reliant e et n pour garantir que d existe
- On peut calculer $d = e^{-1} \pmod{order}$ (théorème de Euler) où $order$ est le nombre d'éléments du groupe
 - $order = n - 1$ si n est premier

RSA: Choix des paramètres

- n et e doivent être publique car ils permettent de chiffrer
- Si n est premier, $order = n - 1$ et on peut facilement calculer $d = e^{-1} \bmod order$
 - Donc n ne doit pas être premier
- On choisit $n = p \times q$ ou p et q sont deux grand nombres premiers
 - Trouver p et q à partir de n est le problème de la factorisation en nombre premier
 - Ce problème est très coûteux à calculer
 - On peut montrer que $order = (p - 1) \times (q - 1)$ lorsque p et q sont premiers
 - p et q restent secret donc un attaquant ne peut pas calculer $order$ à partir de n

RSA: Algorithme

- Generation des clefs:
 - On génère 2 grand nombres premiers p et q
 - On choisit un exposant aléatoire e , ou fixe ($e = 65537$ est souvent utilisé)
 - On doit avoir $\text{pgcd}(e, (p-1) \times (q-1)) = 1$ sinon d n'existe pas
 - La clef publique est le couple $(e, N = p \times q)$
 - On calcule la clef privée $d = e^{-1} \bmod (p-1)(q-1)$
- Chiffrement:
 - On calcule $M = m^e \bmod N$
- Dechiffrement:
 - On calcule $m = M^d \bmod N$

TP: Implémenter RSA

- Dans le fichier `src/crypto/rsa_cipher.py`, en suivant les instructions en commentaire:
- Implémentez la fonction `def keygen() -> dict`
- Implémentez la fonction `def encrypt(msg: str, public_key: tuple) -> int`
- Implémentez la fonction `def decrypt(msg: str, key: dict) -> str`
- Vérifiez que le test `test_rsa()` passe (celui ci peut être un peu long à cause de la génération aléatoire des grands nombres premiers)
- Générez un couple clef publique, clef privé, choisissez un binôme et partagez lui votre clef publique, demandez lui de vous chiffrer un message avec et de vous partager l'entier résultant, déchiffrez le.
- Déchiffrez le message du test `test_rsa_decrypt()`

Signatures digitales

- Permet d'assurer qu'un message a bien été émis ou validé par moi
- Utilise le chiffrement asymétrique: je peux utiliser ma clef privé pour construire une signature digitale (un hash) de la donnée
- Les autres peuvent utiliser ma clef publique pour valider que le hash n'a pu être calculé que par moi
- Exemple: Bitcoin
 - Pour émettre une transaction sur le réseau de type "[mon adresse] envoie X BTC à [adresse de toto]", je signe ce message avec ma clef privé
 - Lorsque les mineurs reçoivent ma transaction, ils vérifient la signature en utilisant ma clef publique (dérivable depuis [mon adresse])
 - Si la signature est validée, ma transaction est intégrée par les mineurs dans un bloc
 - Sinon c'est qu'un attaquant (probablement toto !) a essayé d'émettre cette transaction à ma place pour me voler mes Bitcoins

Protocoles cryptographiques

Introduction

- Un protocole correspond à une recette que plusieurs participants appliquent pour communiquer et atteindre un résultat voulu
- Un protocole cryptographique utilise des primitives cryptographique pour sécuriser le protocole, par exemple:
 - Assurer la confidentialité avec du chiffrement
 - Assurer l'authentification avec des signatures
 - Assurer l'intégrité avec du hashage
- Il existe de nombreux protocoles, on présente ici en surface deux exemple:
 - SSL/TLS
 - Bitcoin
- Pour plus de détails: "Real World Cryptography" de David Wong

Transport sécurisé: SSL/TLS

- Sécurise une communication TCP/IP
- Le protocole le plus utilisé pour sécuriser le web
 - A connu beaucoup de failles
 - Beaucoup de version dépréciées toujours déployées
 - Gros besoin de backward compatibility
- HTTPS correspond à HTTP + TLS
- 2 acteurs:
 - Un client: initie la connection
 - Un serveur: accepte des connections
- 2 phase:
 - Handshake phase: permet d'établir une clef secrète et d'authentifier le serveur
 - Communication chiffrée symetriquement

TLS: Handshake phase

- 3 phases:
 - Negotiation: choix des primitives cryptographique à utiliser
 - TLS est très configurable
 - Si client et serveur ne peuvent se mettre d'accord, la connection est interrompue
 - Echange de clef (avec la primitive choisie pendant la negotiation)
 - Des que l'échange de clef est effectué, le reste de la communication est chiffrée
 - Authentification du serveur
 - Utilisation de l'infrastructure publique de certificats pour assurer que le serveur a qui je parle correspond bien à l'url
 - Pour des échange précis (exemple: app mobile vers un serveur backend précis), on peut assurer l'authentification dans le code client pour plus de sécurité

TLS: Communication chiffrée

- Le chiffrement est fait avec la clef échangée pendant le handshake
- Un nonce est utilisé pour chaque message afin de se protéger des replay et reorder attack

Blockchain et réseaux décentralisés

- Réseaux monétaire (Bitcoin, Dogecoin, ZCash, ...)
- Réseaux applicatif et monétaire (Ethereum, Solana, Cardano, ...)
- Décentralisés: pas de serveur unique central
- Objectif: construire une base de donnée partagée de transactions
 - Appelée blockchain la plupart du temps
- 3 type d'acteurs:
 - Utilisateur: envoie des transactions au réseau
 - Noeud: maintiens une copie de la blockchain, forward les transactions aux noeuds et mineurs
 - Mineur: construit la blockchain
- Un algorithme de consensus est utilisé pour garantir par construction l'unicité de la blockchain sur tout le réseau
 - Doit être sécurisé contre la présence d'acteurs malveillants
 - Algorithme les plus connus: proof of work, proof of stake

Bitcoin: Utilisateur

- Alice génère une paire clef privé + clef publique pour s'authentifier auprès du réseau
 - La clef publique permet de dériver une adresse, qui sera utilisée pour envoyer des bitcoins à Alice
- Elle envoie des transactions signée du type: "J'envoie X bitcoin à l'adresse de Bob et je paye Y de frais"
 - La transaction ne sera acceptée par les mineur que si Alice possède bien X + Y bitcoin
 - La transaction ne sera acceptée que si la signature authentifie bien Alice
 - Il n'y a que elle qui possède sa clef privé, et donc qui peut signer ses messages pour authentifier sa clef publique

Bitcoin: Mineur

- Un mineur reçoit des transactions et tente de construire un bloc a partir d'un certain nombre de transaction
- Le problème est: si tous les mineurs font ça en parallèle, comment obtenir une structure de donnée unique partagée ?

Bitcoin: Proof of Work

- Pour construire un bloc:
 - Le mineur choisit un bloc parent et note son hash (en général le dernier bloc de la chaîne)
 - Le mineur choisit des transactions et les insère dans le bloc
 - Le mineur doit compléter le bloc avec un nombre de sorte à obtenir un hash donné
 - Il va donc essayer en boucle plein de nombres, c'est le proof of work
 - Il envoie ensuite son bloc aux autres mineurs
- A réception d'un bloc:
 - Le mineur vérifie la validité du bloc (transactions et hash)
 - Le mineur intègre son bloc comme fils du bloc parent
- Chaque mineur construit donc un arbre de bloc: la blockchain est la chaîne la plus longue de cet arbre
 - Un bloc est considéré comme final lorsqu'il appartient à la chaîne la plus longue et qu'il a suffisamment de blocs après lui
- Chaque bloc récompense le mineur qui l'a produit, mais cette récompense n'existe que si le bloc est dans la blockchain